How Process Scheduling Works

Study note based on Unit 3: Process Management

Why scheduling is needed

A computer appears to run many programs at once, but a single CPU can execute only one process at a time. Scheduling is how the operating system shares that CPU. When two or more processes compete for it, the OS must decide which one runs next. This decision procedure is called **process scheduling**, and the OS module that makes it is the **scheduler**. Processor scheduling is one of the primary functions of a multiprogramming operating system.

Process states: the backdrop

A process changes state as it runs. The scheduler's work is really about moving processes between these states.

- **New:** the process has been created but not yet admitted to the pool of executable processes.
- **Ready:** the process can run and is waiting for the CPU.
- **Running:** the CPU is executing its instructions. On a single processor, only one process is ever in this state.
- **Waiting (blocked):** the process cannot continue until an event occurs, typically I/O completion.
- **Terminated:** the process has finished, and its memory is de-allocated.

The key transitions are: admitted (New to Ready), scheduler dispatch (Ready to Running), interrupt (Running to Ready), I/O or event wait (Running to Waiting), I/O or event completion (Waiting to Ready), and exit (Running to Terminated).

Scheduling queues

The OS tracks processes using queues of **Process Control Blocks (PCBs)**. A PCB holds everything needed to manage a process: its PID, state, priority, program counter, CPU registers, memory-management data, accounting data, I/O status and a pointer to the next PCB.

- **Ready queue:** all processes that are ready and waiting for the CPU. It is usually a linked list. The queue header holds pointers to the first and last PCBs, and each PCB's pointer field links to the next process in the queue.

- **Device queues:** one per I/O device, holding the processes waiting for that device.

A process's life is a loop. It enters the system and joins the ready queue. When selected, it runs on the CPU. If it needs I/O, it moves to a device queue. When the I/O is served, it returns to the ready queue. This CPU and I/O cycle repeats until the process finishes and exits.

The three types of scheduler

Schedulers are system software modules that handle scheduling at different levels.

	Long-term	Short-term	Medium-term
Also called	Job scheduler	CPU scheduler	Process swapping scheduler
Job	Picks jobs from the pool and loads them into main memory	Picks a ready process and allocates the CPU to it	Removes a process from main memory and reloads it later
Frequency	Low (every few minutes)	High (milliseconds)	Medium, called whenever required
Speed	Slower than short-term	Very fast	Varies with need
Effect on multiprogramming	Controls the degree	Little control	Reduces the degree
In time-sharing systems	Absent or minimal	Minimal	Used

The short-term scheduler runs so often that it must be extremely fast. The long-term scheduler runs rarely, but its choices decide how many processes compete for the CPU at all.

Context switch: the cost of scheduling

When the CPU moves from one process to another, the system must save the state of the old process and load the saved state of the new one. This is a **context switch**. For example, to switch from P0 to P1, an interrupt or system call occurs, the OS saves P0's state into PCB0, then reloads P1's state from PCB1. P0 sits idle until it is switched back in.

The information saved includes the program counter, scheduling information, base and limit register values, the registers in use, the process state, I/O state and accounting data.

Context switch time is pure overhead, because no useful work happens during it. Its speed depends on hardware: memory speed, how many registers must be copied, and whether special instructions exist. Context switching is what lets multiple processes share a single CPU, and it is an essential feature of any multitasking operating system.

Switching between threads of the same process is cheaper than switching between processes, because the threads share the same address space and memory-management information does not need to change.

Putting it together

1. The long-term scheduler admits a job, and it becomes Ready.
2. The short-term scheduler picks it from the ready queue and dispatches it to the CPU.
3. It runs until it finishes, is interrupted, or needs I/O.
4. If interrupted, it goes back to the ready queue. If it needs I/O, it joins a device queue and later returns to Ready.
5. If memory is under pressure, the medium-term scheduler may swap it out and bring it back later.
6. Each change of running process costs a context switch, with state saved to and restored from PCBs.

Quick recap

- Scheduling decides which ready process gets the CPU next.
- The ready queue and device queues are linked lists of PCBs.
- Long-term schedulers admit jobs, short-term schedulers dispatch to the CPU, and medium-term schedulers swap processes in and out.
- A context switch saves one process's state into its PCB and loads another's. It is necessary but is overhead.